



University of Tehran

College of Engineering
School of Electrical & Computer Engineering

Experiment 2

Sessions 3, 4

HDL Clock Generation and Frequency Divider

Digital Logic Laboratory

ECE 045, ECE 895

Laboratory Manual

Spring 1405

Contents

1- Introduction	3
2- Clock Generation using HDL	3
A- Behavioral Modeling using RTL Components	3
B- Using delay statement	4
3- FPGA Design	4
A- Synchronous Counter as a Frequency Divider	5
B- T Flip-Flop	7
C- Display module in FPGA	7
4- Appendices	9

1- Introduction

The goal of this experiment is to introduce the concepts of digital system design using Verilog HDL and FPGA implementation. In the previous experiment, you learned how to generate clock signals using analog oscillator circuits such as the ring oscillator, LM555 timer, and Schmitt trigger oscillator simulated in LTspice. In this experiment, you will transition from analog circuit design to digital hardware description by implementing equivalent clock generation circuits using Verilog HDL.

In this experiment, you will see how you can implement the same circuits in Verilog. There are two ways for implementing such clock circuits in Verilog. You can simply use Verilog delay expressions for setting the ON and OFF duration of the clock pulse. This kind of modeling cannot be synthesized in the synthesis tool but can be simulated in ModelSim which is proper enough for debugging your design. In the other way, you can model the circuit behaviorally using RTL hardware components. This would be a synthesizable model but with lower precision compared to the real hardware. You will then use the Quartus synthesis tool for mapping your designs on Altera Cyclone IV FPGA devices and ModelSim simulation tool for verifying your designs. Then you will use the generated clock signal for a digital circuit, specifically a frequency divider and counter, to observe how clock signals drive sequential digital systems.

By the end of this experiment, you should have learned:

- Verilog HDL basics for clock generation
- Quartus design flow (coding, compilation, simulation)
- ModelSim simulation tool for Verilog designs
- Cyclone IV FPGA implementation and testing
- Frequency divider design using counters and 74 series ICs

2- Clock Generation using Verilog HDL

In the previous section, you have learned the functionality of different clock generation systems by simulating them in LTspice. In this section, you will see how you can implement the same circuits in Verilog. There are two ways for implementing such clock circuits in Verilog.

A- Behavioral Modeling using RTL Components

In this method, you can model the circuit behaviorally using RTL hardware components. This approach produces a synthesizable model that can be implemented on an FPGA, but with lower precision compared to the real hardware. To observe this fact, a Verilog code named LM555_timer.v for the LM555 timer circuit is provided to you. Follow the instructions below:

- 1- Before simulating the design, first read the code completely and understand what is going on.
- 2- Draw a block diagram of the design in your report and explain the hardware and the functionality.

- 3- To verify this design, write a simple testbench. Make an instance of the design in the testbench and test the design by providing the parameters. Measure the duty cycle of this circuit by setting the resistor values from experiment 1 (1k Ω , 10k Ω , and 100k Ω for R2). Include the waveforms of the inputs and outputs in your report. Are the results of these two sections completely matched? Explain the reason if not.

B- Using Delay Statements

You have seen the functionality of a ring oscillator by simulating the circuit in LTspice. Now you will redo the simulation using Verilog HDL code. In this alternative method, you will use Verilog delay expressions for setting the ON and OFF duration of the clock pulse. This kind of modeling cannot be synthesized in the synthesis tool but can be simulated in ModelSim, which is suitable enough for debugging your design.

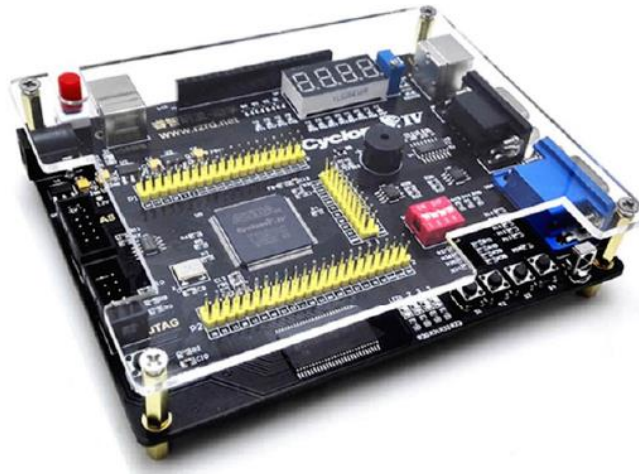
- 1- Write a Verilog description of the ring oscillator. For this purpose you need to use an odd number of inverters. Your design must be parameterized so that it receives the delay value of the inverters and the number of inverters as parameters and generates the corresponding oscillator as the output.
- 2- To verify your design, you need a simple testbench. Make an instance of the ring oscillator in your testbench and test the design by providing the parameters. For real modeling of the ring oscillator, in your testbench use the delay values that you calculated in experiment 1.
- 3- Measure the ring oscillator frequency and include the waveforms of the inputs and outputs in your report.

3- FPGA Design

In this part, you will be familiar with FPGA design. First you will start with simulating different design using 74series IC. In this part you will use Intel Quatus Synthesis tool for mapping your designs on Altera CycloneIV FPGA devices and Modelsim simulation tool for verifying your designs. The FPGA device that you will use in this and all other experiments until the end of the semester is Cyclone IV device. Figure 1 shows the layout of the board. In this section you use 74 series ICs. These ICs are placed in Quartus tool inside the Altera MAXPLUS2 libraries. In order to access these libraries and for simplicity, you will design the circuits in a block diagram format. Appendix A shows a guideline for building a schematic design project in Quartus software.

In this part you will use the clock circuits of the section 2.B along with FPGA to simulate a clock frequency divider. There are many parts which should be considered in designing the desired system. Below is a short description of these parts and what you should follow for this experiment.

Figure 1. Altera cyclone IV board



A- Synchronous Counter as a Frequency Divider

Different clock signals can be produced by the aforementioned methods, but not all of them are suitable for all applications. Consider a 1 Hz clock signal which can be easily produced by a LM555 timer. The timing error of this signal can be 1-2%, which is too much for a low frequency like this, while this error range is acceptable for higher frequencies. So, a higher frequency can be chosen and then a frequency divider can reduce the frequency to the desired one.

Counters can be used as a frequency divider. 74LS191 is a synchronous 4-bit up/down counter. As figure 2 shows, it has 4 inputs D-A (most to least), 4 outputs QD-QA (most to least) and a parallel load, which allows for presetting an initial value for the counter. With this pin layout two counters can be cascaded when the modulus is more than 4 bits.

The timing diagram of 74191 is shown in figure 3. When up counting is desired the initial value is obtained by:

$$\text{Initial value} = \text{Maximum value} - \text{Modulus}$$

Construct a divide by 113 synchronous up-counter as shown in figure 4. You should:

- 1- Use the MAXPLUS2 74 series (74191) IC in your block diagram and connect them based on figure 4 and layout of figure 2.
- 2- A Presetting mechanism is necessary for initial loading of the counters of figure 4. The mechanism as you have learned in the logic design course can include an AND gate for ANDing a preset input signal with the load inputs of the counter. Perform this presetting by using 7408 AND gate of MAXPLUS2 74 series.

- 3- Write a testbench for the design and set the required input values like the initial load value, enable and up/count signal. For counting up the DNU signal must be set to zero.
- 4- Use the ring oscillator of the section 2.B as the clock input of the LSB counter. Since the ring oscillator model is not a synthesizable design, connect the ring oscillator inside the testebench and not in the Quartus block diagram.
- 5- After simulating the design in Modelsim software, record the results of carry out of the max/min signal of the MSB counter and measure its frequency, compare the results with the frequency of the input clock.

Figure 2. Pin layout of 74191

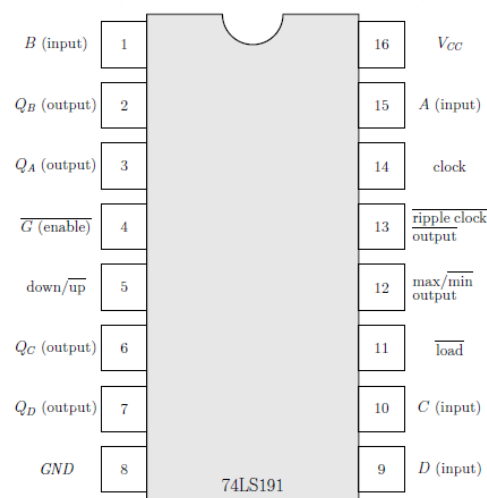


Figure 3. Timing diagram of 74191

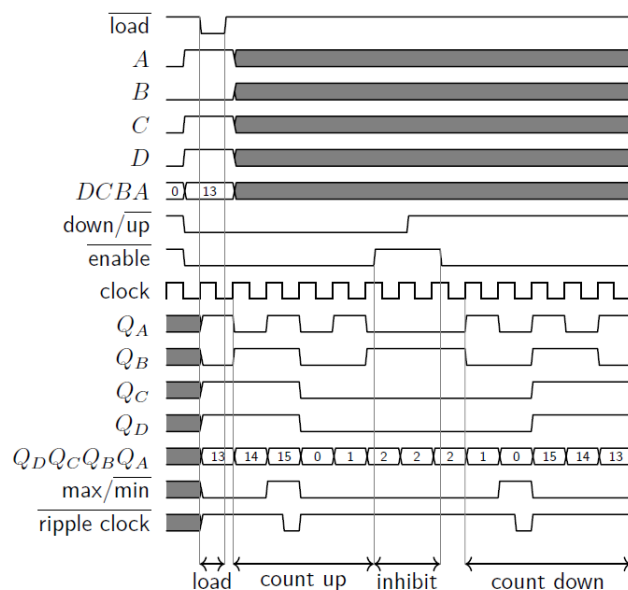
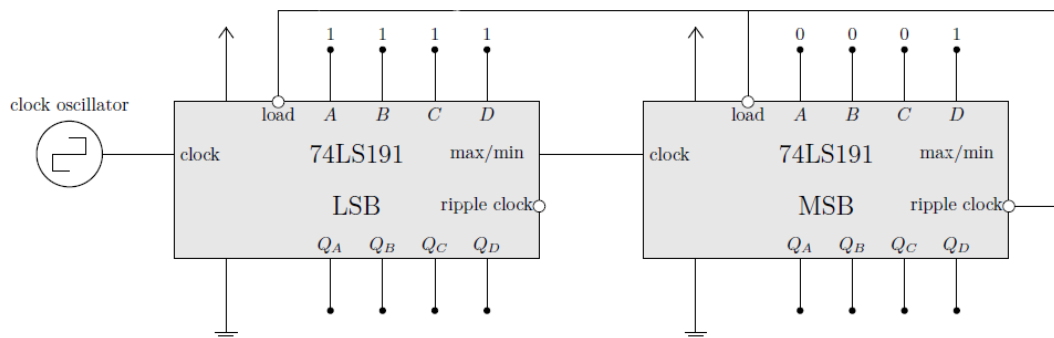


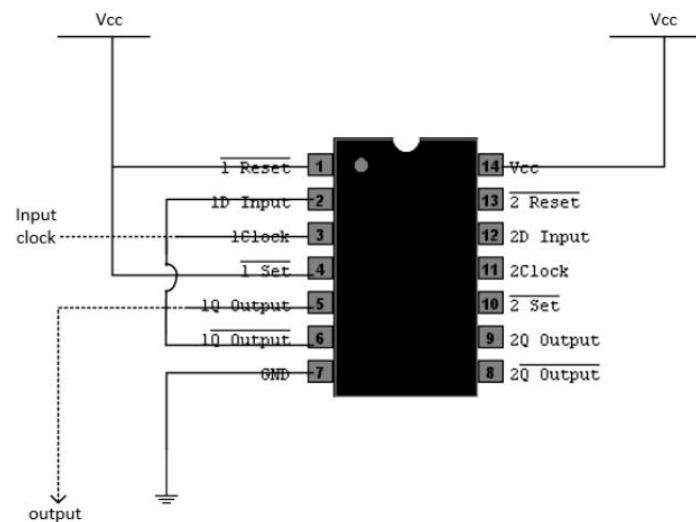
Figure 4. Frequency divider using 74191



B- T Flip-Flop

You should use a T Flip-Flop after counter to produce a 50 percent duty cycle signal. Use 7474 IC from the MAXPLUS2 library for this purpose. 7474 is a dual D Flip-Flop that can be converted to a T Flip-Flop if it is used as in figure 5.

Figure 5. 7474 as a T Flip-Flop



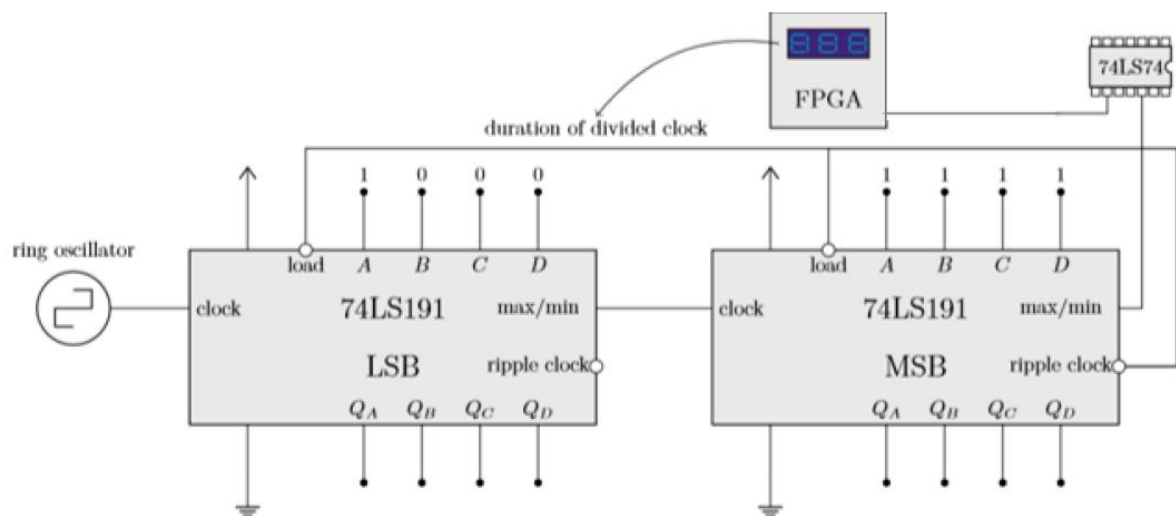
C- Display module in FPGA

As the output of ring oscillator is a high frequency signal, it may be difficult to observe it on an analog oscilloscope. Alternatively, you can use a FPGA and calculate the frequency of ring oscillator. To do this, the output of the counter, after passing a T flip-flop, will be connected as the input of the FPGA. The Verilog code for the display module is provided to you (*display.v*). This module counts the number of

clock cycles that the input signal is on which is called duration. The duration indicates the divider divisor value.

- 1- Add the Verilog code *display.v* to your project.
- 2- Set this module as the top-level entity and synthesize it.
- 3- Generate a schematic symbol for this module.
- 4- Set back the Clock adjusting block diagram as the top-level entity.
- 5- Now connect these circuits together according to figure 6. Connect the display module block diagram to the proper pins of the counter.
- 6- Write a testbench for this design and set the required inputs and outputs and test your design in ModelSim and check if it is working correctly.
- 7- Calculate the ring oscillator frequency based on the results you have recorded. You should calculate the frequency of the divided signal based on the number you observed on the duration signal and then calculate the frequency of ring oscillator.

Figure 6. Clock adjusting system



4- Appendices

Appendix A Using Quartus II

A.1 Create the Project

- 1- Click on File ▷ New Project Wizard
- 2- Create an appropriate directory for your project and complete the form
- 3- Select the FPGA device as Cyclone EP2C20F484C7 and then click Finish
- 4- From File ▷ New select the Verilog HDL File
- 5- Write your Verilog code in this window
- 6-

A.2 Compilation

- Select Processing ▷ Start Compilation
- Click on quick shortcut ►

There may be lots of warnings and some errors after compilation. The warnings are not so important while the errors should be completely removed.

A.3 Pin Assignment

After you successfully compile your design, you should set your design with physical pins of the Cyclone II FPGA on DE1 board. You can find the position and the index of each pin from the DE1 user manual.

From Assignments ▷ Pin Planner, *Pin Planner window* will appear and you can select the corresponding locations from the list. When you are done with this, recompile your project.

A.4 Program the Design

- 1- Click on the  icon on the toolbar to open Programmer
- 2- Choose USB Blaster from the Hardware setup in the new window
- 3- Click on the start button

Your design will be programmed on the board when it is finished.

A.5 Examine the Timing and Resources

Now you can examine the resource usage of your design from Processing ▷ Compilation Report or by clicking on quick shortcut  , and the timing from the Tools ▷ TimeQuest Timing Analyzer.

Figure 7: Creating new project

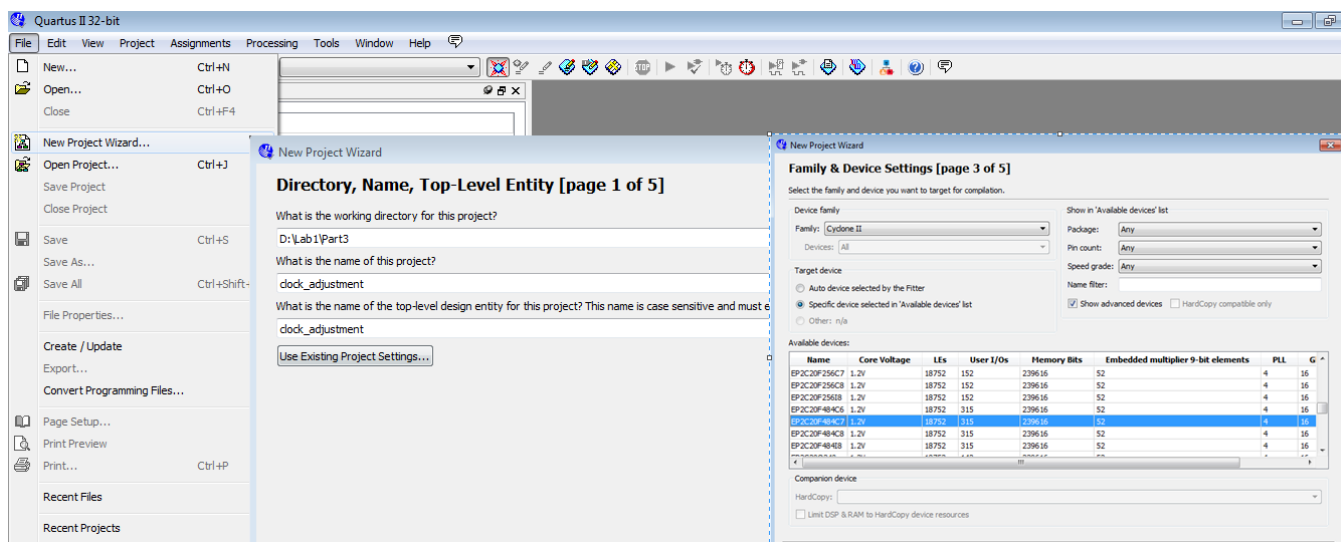


Figure 8: Writing Verilog HDL code

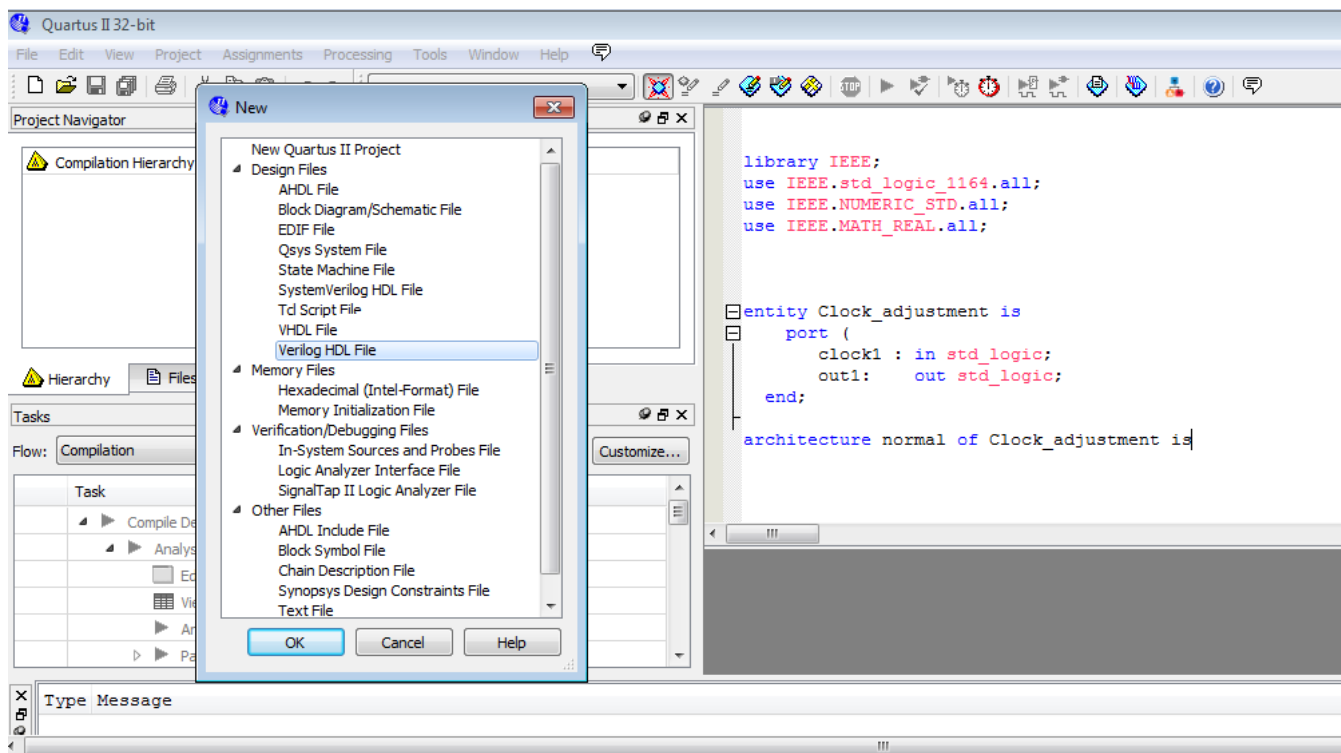


Figure 9: Pin Planner window

